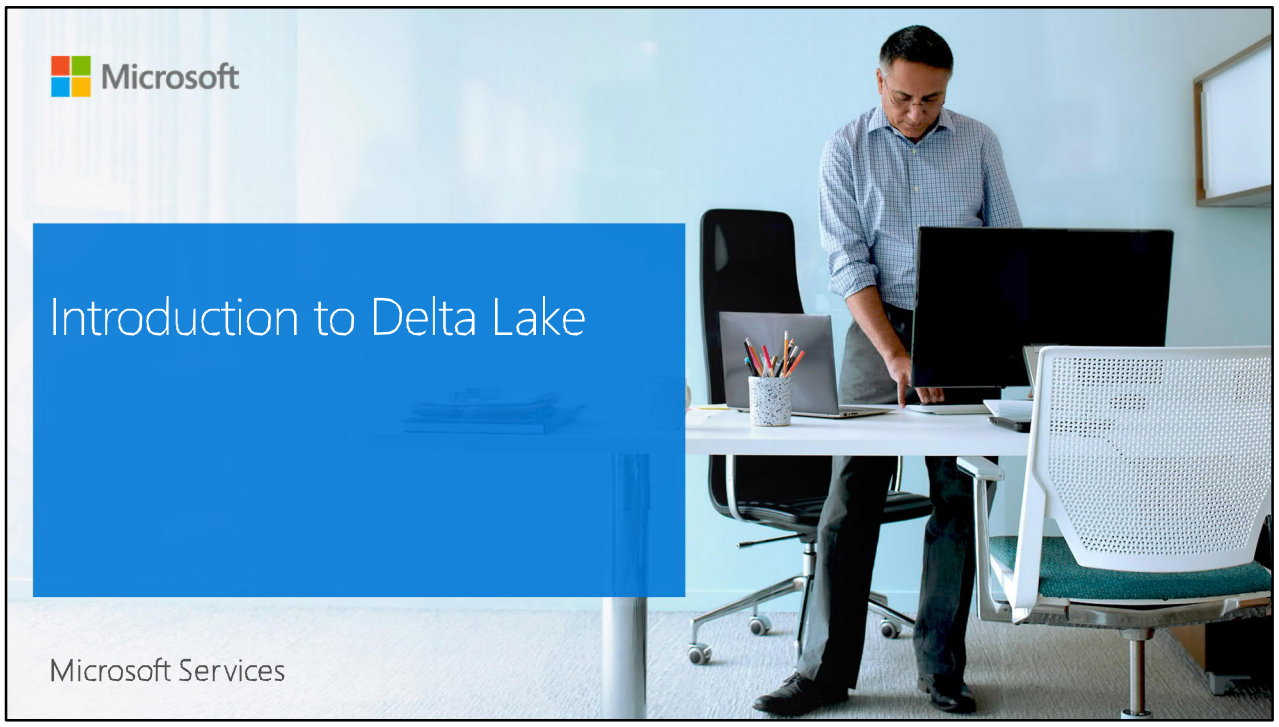

Microsoft
Introduction to Delta Lake
Microsoft Services

Spark Shortcomings

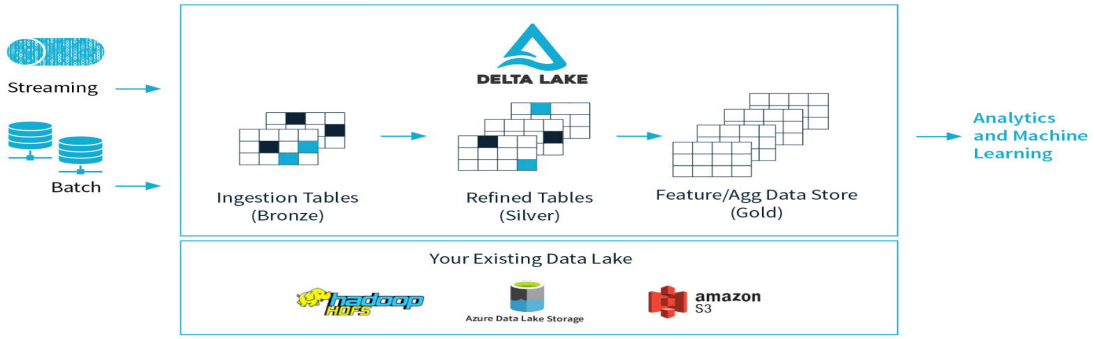
- Basic Schema Inference/Enforcement (file level schema understanding only, no global schema): Many formats (CSV, JSON) allow silent type coercion or nulls
- Extremely expensive upserts (updates/merge)
- Once you overwrite data, the old version is not recoverable. Impossible to audit or recover
- No Transactions
- Frequent/Small File Problem

Data teams are typically facing the following challenges:

- Hard to append data
- Modification of existing data is difficult
- Jobs failing mid way
- Real-time operations (*Mixing streaming and batch leads to inconsistency)*
- Costly to keep historical versions of the data (*Regulated environments require reproducibility, auditing, governance)*
- Difficult to handle large metadata (*For large data lakes the metadata itself becomes difficult to manage)*
- "Too many files" problems (*Data lakes are not great at handling millions of small files)*
- Hard to get great performance (*Partitioning the data for performance is error-prone and difficult to change)*
- Data quality issues (*It's a constant headache to ensure that all the data is correct and high quality)*

Delta Lake

- Delta Lake is a storage layer that brings scalable, ACID transactions to Apache Spark and other big-data engines.
- Delta Lake runs on top of your existing [cloud storage] and is fully compatible with Apache Spark APIs



Delta Lake is an open source storage layer that brings reliability to data lakes. Delta Lake provides ACID transactions, scalable metadata handling, and unifies streaming and batch data processing. Delta Lake runs on top of your existing data lake and is fully compatible with Apache Spark APIs.
Specifically, Delta Lake offers:
•ACID (Atomicity, Concurrency, Isolation, Durability) transactions on Data Lake: Serializable isolation levels ensure that readers never see inconsistent data.
•Scalable metadata handling: Leverages Spark's distributed processing power to handle all the metadata for petabyte-scale tables with billions of files at ease.
•Streaming and batch unification: A table in Delta Lake can be a batch as well as a streaming source and sink. Streaming data ingest, batch historic backfill, interactive queries all just work out of the box.
•Schema enforcement: Automatically handles schema variations to prevent insertion of bad records during ingestion.
•Time travel: Data versioning enables rollbacks, full historical audit trails, and reproducible machine learning experiments.
•Upserts and deletes: Supports merge, update and delete operations to enable complex use cases like change-data-capture, slowly-changing-dimension (SCD) operations, streaming upserts, and so on.

https://docs.delta.io/latest/delta-intro.html

Delta lake (cont...)

- Open-source software that extends Parquet data files with a file-based transaction log for ACID transactions and scalable metadata handling (data types, columns, etc.).
- Databricks originally developed the Delta Lake protocol and continues to actively contribute to the open source project.
- Delta Lake is the default format for all operations on Databricks.

https://learn.microsoft.com/en-us/azure/databricks/delta/tutorial

Delta Lake Overview

- ACID transactions (multiple concurrent reads and writes)
- Open format (Parquet)
- Scalable meta-data handling
- Streaming and batch unification
- Schema enforcement
- Time travel
- Upserts and deletes
- Audit history
- 100% compatible with Apache Spark API

Key Features

ACID Transactions:
Data lakes typically have multiple data pipelines reading and writing data concurrently, and data engineers have to go through a tedious process to ensure data integrity, due to the lack of transactions. Delta Lake brings ACID transactions to your data lakes. It provides serializability, the strongest level of isolation level. Learn more at Diving into Delta Lake: Unpacking the Transaction Log.
Scalable Metadata Handling:
In big data, even the metadata itself can be "big data". Delta Lake treats metadata just like data, leveraging Spark's distributed processing power to handle all its metadata. As a result, Delta Lake can handle petabyte-scale tables with billions of partitions and files at ease.
Time Travel (data versioning):
Delta Lake provides snapshots of data enabling developers to access and revert to earlier versions of data for audits, rollbacks or to reproduce experiments. Learn more in Introducing Delta Lake Time Travel for Large Scale Data Lakes.
Open Format:
All data in Delta Lake is stored in Apache Parquet format enabling Delta Lake to leverage the efficient compression and encoding schemes that are native to Parquet.
Unified Batch and Streaming Source and Sink:

A table in Delta Lake is both a batch table, as well as a streaming source and sink. Streaming data ingest, batch historic backfill, and interactive queries all just work out of the box.

Schema Enforcement:

Delta Lake provides the ability to specify your schema and enforce it. This helps ensure that the data types are correct and required columns are present, preventing bad data from causing data corruption. For more information, refer to Diving Into Delta Lake: Schema Enforcement & Evolution.

Schema Evolution:

Big data is continuously changing. Delta Lake enables you to make changes to a table schema that can be applied automatically, without the need for cumbersome DDL. For more information, refer to Diving Into Delta Lake: Schema Enforcement & Evolution.

Audit History:

Delta Lake transaction log records details about every change made to data providing a full audit trail of the changes.

Updates and Deletes:

Delta Lake supports Scala / Java APIs to merge, update and delete datasets. This allows you to easily comply with GDPR and CCPA and also simplifies use cases like change data capture. For more information, refer to Announcing the Delta Lake 0.3.0 Release and Simple, Reliable Upserts and Deletes on Delta Lake Tables using Python APIs which includes code snippets for merge, update, and delete DML commands.

100% Compatible with Apache Spark API:

Developers can use Delta Lake with their existing data pipelines with minimal change as it is fully compatible with Spark, the commonly used big data processing engine.

https://delta.io/

Spark + Delta Lake

- Schema Enforcement: Detects invalid data, allows schema evolution, column updates (upcasting)
- Transactions
- Transaction Log (the Delta Log). A write is only "committed" once it's fully successful.
  - If it fails, partial files are automatically ignored
- Upserts: provides a highly efficient MERGE command.
- Time Travel/Data Versioning/Data History
- Much more

https://delta.io/learn/getting-started/

You can think about Delta as a file format that your engine can leverage to bring the following capabilities out of the box:

- ACID transactions
- Support for DELETE/UPDATE/MERGE
- Unify batch & streaming
- Time Travel
- Cloning Tables (clone zero copy)
- Generated partitions
- CDF - Change Data Flow (Databricks runtime)
- Blazing-fast queries

Databricks Delta Lake Overview

- Delta Lake uses versioned Parquet files to store your data in your cloud storage.
- Delta Lake also stores a transaction log to keep track of all the commits made to the table or blob store directory to provide ACID transactions

Refer to these links for review material related to Databricks Delta. Basically, Databricks Delta optimizes storage of data in the Databricks File System through the use of Delta tables which use the Parquet format as their underlying file format.

https://docs.azuredatabricks.net/delta/delta-intro.html#delta-intro
https://docs.azuredatabricks.net/delta/optimizations.html

Delta Lake Transaction Logs

- Delta Lake brings ACID transactions to data lakes by combining immutable Parquet data files with a transaction log (_delta_log).
- Ordered, append-only record of every change ever made to a Delta table. (<table_path>/_delta_log/)
- Each commit is written out as a JSON file, starting with 000000.json
  - Additional changes to the table generate subsequent JSON files in ascending numerical order so that the next commit is written out as 000001.json, the following as 000002.json, and so on.

https://www.databricks.com/blog/2019/08/21/diving-into-delta-lake-unpacking-the-transaction-log.html

What Is the Delta Lake Transaction Log?
The Delta Lake transaction log (also known as the DeltaLog) is an ordered record of every transaction that has ever been performed on a Delta Lake table since its inception.

What Is the Transaction Log Used For?

1. Single Source of Truth

Delta Lake is built on top of Apache Spark™ in order to allow multiple readers and writers of a given table to all work on the table at the same time. In order to show users correct views of the data at all times, the Delta Lake transaction log serves as a single source of truth - the central repository that tracks all changes that users make to the table.

When a user reads a Delta Lake table for the first time or runs a new query on an open table that has been modified since the last time it was read, Spark checks the transaction log to see what new transactions have posted to the table, and then updates the end user's table with those new changes. This ensures that a user's version of a table is always synchronized with the master record as of the most recent query, and that users cannot make divergent, conflicting changes to a table.

2. The Implementation of Atomicity on Delta Lake

One of the four properties of ACID transactions, atomicity, guarantees that operations (like an INSERT or UPDATE) performed on your data lake either complete fully, or don't complete at all.

The transaction log is the mechanism through which Delta Lake is able to offer the guarantee of atomicity. For all intents and purposes, if it's not recorded in the transaction log, it never happened. By only recording transactions that execute fully and completely, and using that record as the single source of truth, the Delta Lake transaction log allows users to reason about their data, and have peace of mind about its fundamental trustworthiness, at petabyte scale.

3. How Does the Transaction Log Work?

Whenever a user performs an operation to modify a table (such as an INSERT, UPDATE or DELETE), Delta Lake breaks that operation down into a series of discrete steps composed of one or more of the actions below.

- Add file - adds a data file.
- Remove file - removes a data file.
- Update metadata - Updates the table's metadata (e.g., changing the table's name, schema or partitioning).
- Set transaction - Records that a structured streaming job has committed a micro-batch with the given ID.
- Change protocol - enables new features by switching the Delta Lake transaction log to the newest software protocol.
- Commit info - Contains information around the commit, which operation was made, from where and at what time.

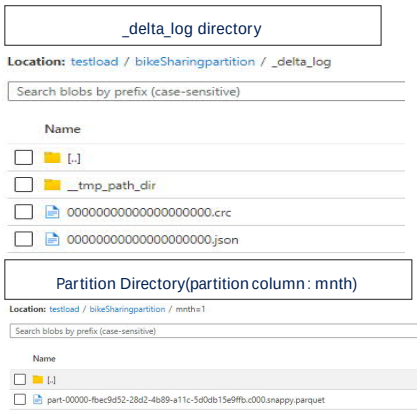
Those actions are then recorded in the transaction log as ordered, atomic units known as commits.

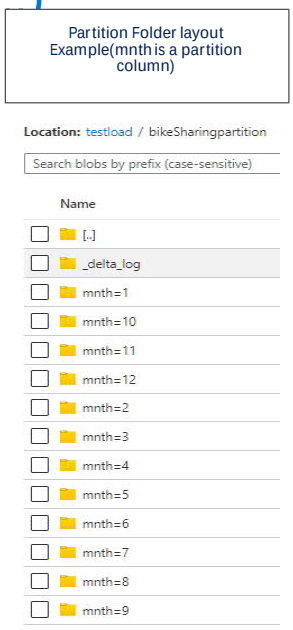
For example, suppose a user creates a transaction to add a new column to a table plus add some more data to it. Delta Lake would break that transaction down into its component parts, and once the transaction completes, add them to the transaction log as the following commits:

- Update metadata - change the schema to include the new column
- Add file - for each new file added

Delta Lake Transaction Log (cont..)

- Entries in the log, called *delta files*, are stored as atomic collections of actions in the _delta_log directory, at the root of a table
- Entries in the log are encoded using JSON and are named as zero-padded contiguous integers.



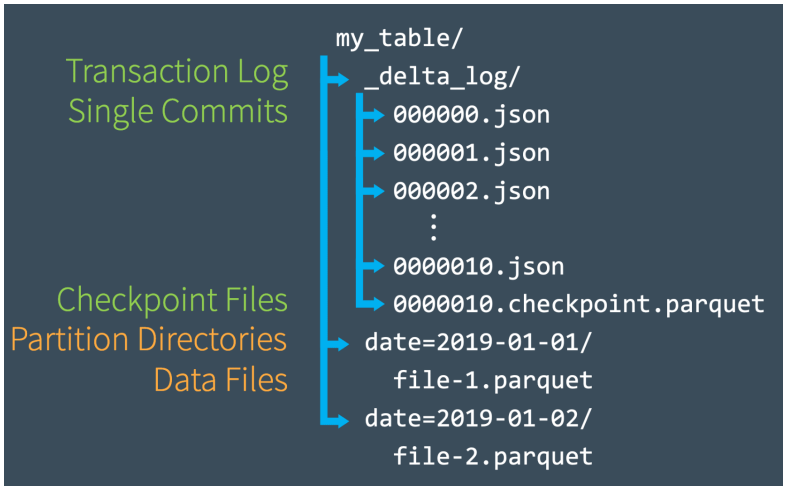


https://databricks.com/blog/2019/08/21/diving-into-delta-lake-unpacking-the-transaction-log.html

Walk through pictorial representation of _delta_log file, partition folder contents and partion root folder layout.

Delta Lake Transaction Log (cont…)

- Delta Lake automatically generates checkpoint files every 10 commits
- These checkpoint files save the entire state of the table at a point in time – in native Parquet format that is quick and easy for Spark to read



https://databricks.com/blog/2019/08/21/diving-into-delta-lake-unpacking-the-transaction-log.html

When a user creates a Delta Lake table, that table's transaction log is automatically created in the _delta_log subdirectory. As he or she makes changes to that table, those changes are recorded as ordered, atomic commits in the transaction log. Each commit is written out as a JSON file, starting with 000000.json. Additional changes to the table generate subsequent JSON files in ascending numerical order so that the next commit is written out as 000001.json, the following as 000002.json, and so on.

So, as an example, perhaps we might add additional records to our table from the data files 1.parquet and 2.parquet. That transaction would automatically be added to the transaction log, saved to disk as commit 000000.json. Then, perhaps we change our minds and decide to remove those files and add a new file instead (3.parquet). Those actions would be recorded as the next commit in the transaction log, as 000001.json, as shown above.

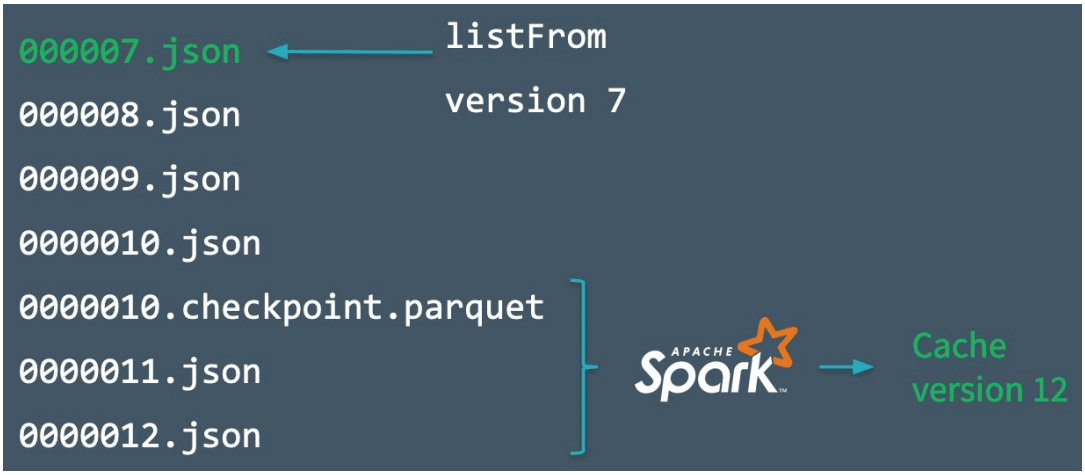
Even though 1.parquet and 2.parquet are no longer part of our Delta Lake table, their addition and removal are still recorded in the transaction log because those operations were performed on our table - despite the fact that they ultimately canceled each other out. Delta Lake still retains atomic commits like these to ensure that in the event we need to audit our table or use "time travel" to see what our table looked like at a given point in time, we could do so accurately..

Quickly Recomputing State With Checkpoint Files

Once we've made several commits to the transaction log, Delta Lake saves a checkpoint file in Parquet format in the same _delta_log subdirectory. Delta Lake automatically generates checkpoint as needed to maintain good read performance.

These checkpoint files save the entire state of the table at a point in time - in native Parquet format that is quick and easy for Spark to read. In other words, they offer the Spark reader a sort of "shortcut" to fully reproducing a table's state that allows Spark to avoid reprocessing what could be thousands of tiny, inefficient JSON files.

Delta Lake Transaction Log (cont...)



To get up to speed, Spark can run a listFrom operation to view all the files in the transaction log, quickly skip to the newest checkpoint file, and only process those JSON commits made since the most recent checkpoint file was saved.

To demonstrate how this works, imagine that we've created commits all the way through 000007.json as shown in the diagram below. Spark is up to speed through this commit, having automatically cached the most recent version of the table in memory.

In the meantime, though, several other writers (perhaps your overly eager teammates) have written new data to the table, adding commits all the way through 0000012.json.

To incorporate these new transactions and update the state of our table, Spark will then run a listFrom version 7 operation to see the new changes to the table.

Rather than processing all of the intermediate JSON files, Spark can skip ahead to the most recent checkpoint file, since it contains the entire state of the table at

commit #10. Now, Spark only has to perform incremental processing of 0000011.json and 0000012.json to have the current state of the table. Spark then caches version 12 of the table in memory. By following this workflow, Delta Lake is able to use Spark to keep the state of a table updated at all times in an efficient manner.

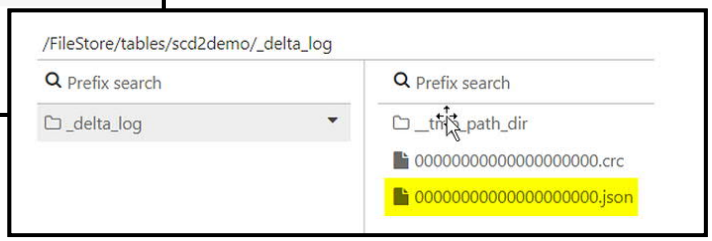
Delta Lake Transaction Log (cont...)

```
1  %sql
2  CREATE OR REPLACE TABLE scd2demo(
3  pk1 INT,
4  pk2 STRING,
5  dim1 INT,
6  dim2 int,
7  dim3 int,
8  dim4 int,
9  active_status STRING,
10 start_date TIMESTAMP,
11 end_date TIMESTAMP)
12 LOCATION 'FileStore/tables/scd2demo'
```

▸ (4) Spark Jobs

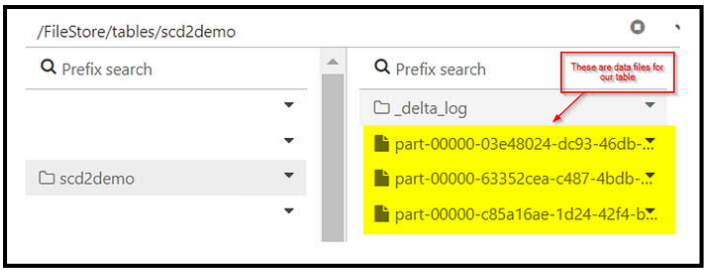
▸ _sqldf: pyspark.sql.dataframe.DataFrame

OK



Delta Lake Transaction Log (cont...)

```
%sql
insert into scd2demo values(111,'Unit1',200,300,400,500,'Y',current_timestamp(),'9999-12-31');
insert into scd2demo values(222,'Unit2',200,300,400,500,'Y',current_timestamp(),'9999-12-31');
insert into scd2demo values(333,'Unit3',200,300,400,500,'Y',current_timestamp(),'9999-12-31');
```



Schema Enforcement/Validation

- Safeguard in Delta Lake that ensures data quality by rejecting writes to a table that do not match the table's schema
- Checks to see whether each column in data being inserted into the table is on its list of expected columns
- Cannot contain any additional columns that are not present in the target table's schema
- Cannot have column data types that differ from the column data types in the target table
- Can not contain column names that differ only by case

To determine whether a write to a table is compatible, Delta Lake uses the following rules. The DataFrame to be written:

- Cannot contain any additional columns that are not present in the target table's schema. Conversely, it's OK if the incoming data doesn't contain every column in the table - those columns will simply be assigned null values.
- Cannot have column data types that differ from the column data types in the target table. If a target table's column contains StringType data, but the corresponding column in the DataFrame contains IntegerType data, schema enforcement will raise an exception and prevent the write operation from taking place.
- Can not contain column names that differ only by case. This means that you cannot have columns such as 'Foo' and 'foo' defined in the same table. While Spark can be used in case sensitive or insensitive (default) mode, Delta Lake is case-preserving but insensitive when storing the schema. Parquet is case sensitive when storing and returning column information. To avoid potential mistakes, data corruption or loss issues (which we've personally experienced at Databricks), we decided to add this restriction.

Schema Validation

```
# Show original DataFrame's schema
original_loans.printSchema()

"""
root
  |-- addr_state: string (nullable = true)
  |-- count: integer (nullable = true)
"""

# Show new DataFrame's schema
loans.printSchema()

""" Returns:
root
  |-- addr_state: string (nullable = true)
  |-- count: integer (nullable = true)
  |-- amount: double (nullable = true) # new column
"""
```

Schema Validation

```
# Attempt to append new DataFrame (with new column) to existing table
loans.write.format("delta").mode("append").save(DELTALAKE_PATH)

""" Returns:
<span style="color: red;">
A schema mismatch detected when writing to the Delta table.

To enable schema migration, please set:
'.option("mergeSchema", "true")\'
```

Delta Lake enforces the schema and stops the write from occurring. To help identify which column(s) caused the mismatch, Spark prints out both schemas in the stack trace for comparison.

Schema Evolution

```
# Add the mergeSchema option
loans.write.format("delta") \
           .option("mergeSchema", "true") \
           .mode("append") \
           .save(DELTALAKE_SILVER_PATH)
```

you can set this option for the entire Spark session by adding spark.databricks.delta.schema.autoMerge = True to your Spark configuration. Use with caution, as schema enforcement will no longer warn you about unintended schema mismatches.

By including the mergeSchema option in your query, any columns that are present in the DataFrame but not in the target table are automatically added on to the end of the schema as part of a write transaction. Nested fields can also be added, and these fields will get added to the end of their respective struct columns as well.

The following types of schema changes are eligible for schema evolution during table appends or overwrites:

- Adding new columns (this is the most common scenario)
- Changing of data types from NullType -> any other type, or upcasts from ByteType -> ShortType -> IntegerType

Other changes, which are not eligible for schema evolution, require that the schema and data are overwritten by adding .option("overwriteSchema",

"true").
For example, in the case where the column "Foo" was originally an integer data type and the new schema would be a string data type, then all of the Parquet (data) files would need to be re-written.
Those changes include:

- Dropping a column
- Changing an existing column's data type (in place)
- Renaming column names that differ only by case (e.g. "Foo" and "foo")